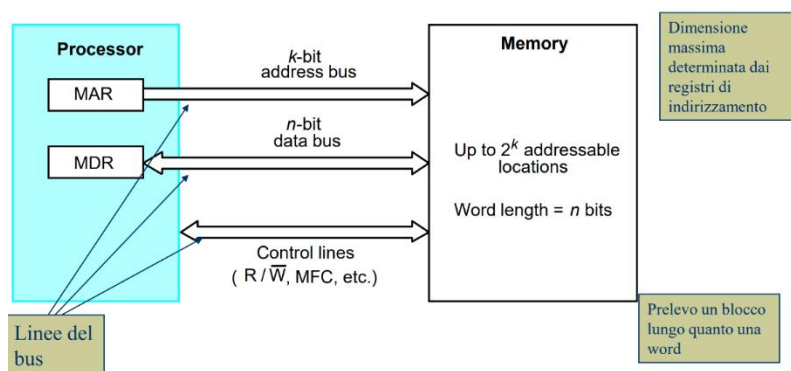


Sommario

1.0 Memoria	1
1.1 cache miss.....	1
1.2 Memoria Cache e Memoria virtuale	3
1.2.3 Memorie statiche e memorie dinamiche	6
1.2.4 Sviluppi Tecnologiche	10
1.2.5 Cache Virtuale	10
Lezione 7 maggio Architetture parallele	12
2.1 Architetture SIMD	14
2.2 Architetture MultiCore e Multithreading	15
2.2.1 Legge di Amdahl	15
RICAPITOLAZIONE	19
FASE 1: Singolo modulo SIMD con 4 lane	19
FASE 2: ho 4 moduli SIMD senza thread.....	19
FASE3: 4 moduli SIMD con thread.....	19

1.0 Memoria

1.1 cache miss



2

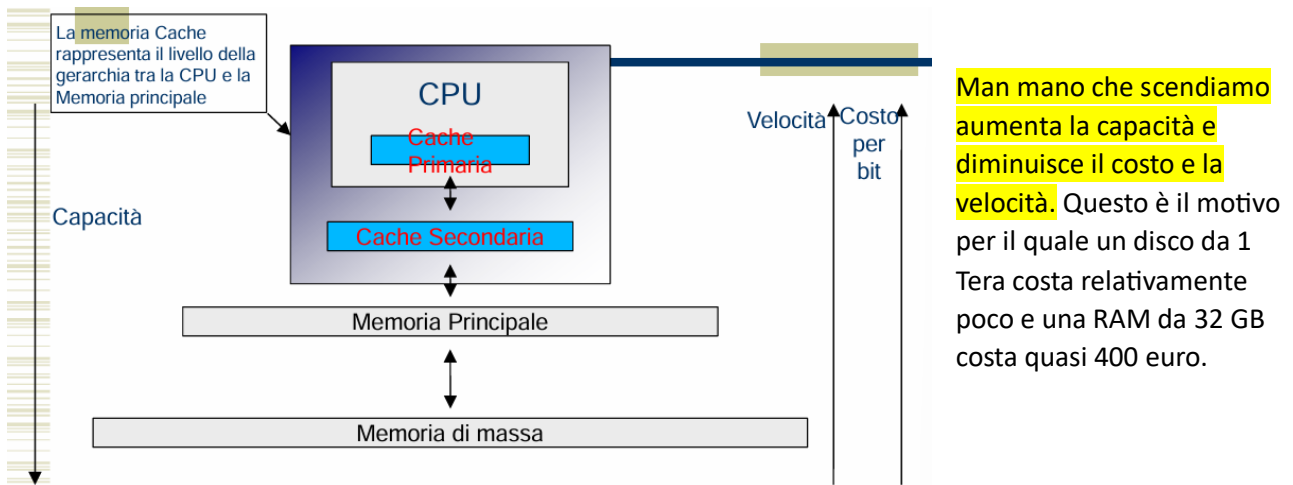
A causa del ruolo critico che la memoria assume quando si cerca un incremento nel throughput del sistema, essa viene detta: **The Von Neumann Bottleneck**

Un modo per incrementare le prestazioni del sistema è quello di aumentare opportunamente la banda della memoria (bps accessibili). Tale incremento di banda è ottenibile in tre modi:

1. Aumentando la frequenza del clock di memoria;
2. Aumentando la larghezza della word e quindi accedendo a più bit per ciclo;

3. Replicando i moduli di memoria in accesso concorrente (simile alla precedente);

la soluzione che si adotta è la prima ovvero aumentare la frequenza della memoria. Questo significa però che i costi si alzano e le dimensioni si riducono.



La cpu lavora a frequenza elevatissime rispetto alla memoria e di conseguenza la cache. A questo punto utilizzare una logica di accesso sequenziale oppure software risulterebbe inefficiente. Il tutto avviene quindi in modo quasi istantaneo tramite dei circuiti logici cablati.

Quando la CPU deve prendere un dato, prende il Tag dell'indirizzo e viene inviato ai comparatori:

- Se il comparatore trova una corrispondenza esatta restituisce un livello logico alto segnalando un **Cache Hit**
- Altrimenti se nessun comparatore risponde in modo affermativo si verifica un **cache Miss**.

Le uscite dei comparatori terminano in una porta NOR. Questo perché bisogna restituire valore logico alto se c'è cache miss e valore logico basso se c'è cache hit, La porta NOR quindi restituisce 1 se tutti i comparatori sono 0 e quindi cache miss, altrimenti se almeno un comparatore dà esito positivo viene restituito 0.

Abbiamo alcuni parametri importanti, molti incontrati già nelle lezioni precedenti:

- **Hit Time** è il tempo misurato in cicli di clock necessario per caricare un elemento che si trova in cache. Ovvero quando si verifica un cache Hit il tempo per prendere quel dato. Generalmente 1 ciclo di clock;
- **Miss penalty**: come detto se non troviamo un elemento in cache abbiamo un cache miss. Questo parametro rappresenta il numero di cicli di clock necessari per caricare l'elemento dalla RAM alla cache
- **Access Time**: tempo di accesso alla memoria principale o a una cache di livello superiore (per livello superiore si intende la cache meno veloce)
- **Transfer time**: tempo di trasferimento di un blocco in cui è presente l'elemento dalla RAM alla memoria cache.
- **Cache size**: dimensione della cache, qualche Mbyte
- **Tempo di accesso medio** = $\text{Tempo di Hit} * \text{Percentuale di hit} + \text{Tempo di miss} * (1 - \text{Percentuale di hit})$. Il tempo di miss generalmente è 10 volte il tempo di hit.

L'uscita della NOR è il flag di cache miss che innesca l'hardware del controller della memoria che congela la pipe e avvia il recupero del blocco nella RAM.

Gli algoritmi di sostituzione possono essere o randomici oppure LRU. Per LRU si sfruttano i bit del contatore. Se c'è un hit il contatore del blocco dell'hit viene azzerato e gli altri blocchi se hanno valore inferiori del blocco hittato vengono incrementati, gli altri rimangono invariati. Mentre se c'è un miss viene inserito il nuovo blocco con contatore=0 e gli altri incrementati. Quando c'è un miss e i blocchi sono tutti pieni si andrà a sostituire il blocco con il contatore più alto.

Le operazioni di lettura sono predominanti sulle operazioni di scrittura (tutte le operazioni su istruzioni sono solo operazioni di lettura).

Nel caso di un'operazione di lettura, in parallelo avvengono due cose: confronto del Tag e lettura del dato. Potremmo pensare ma leggi il dato senza sapere se un comparatore ti ha risposto positivamente? È vero ma se c'è un HIT ho il dato già pronto, se c'è un Miss l'hardware ignora semplicemente quel dato. Leggere un dato non modifica nulla per questo si fa.

Nel caso di una operazione di scrittura, non è possibile cominciare l'operazione finché non si ha un'hit perché altrimenti se ho miss andrei a scrivere in un'area in cui non dovevo scrivere. Visto che l'operazione di accesso all'area dati della cache, non può avvenire in parallelo con il confronto delle label, in genere, un'operazione di scrittura impiega più tempo di una di lettura. Per le operazioni di scrittura esistono i due meccanismi già affrontati nella scorsa lezione.

1.2 Memoria Cache e Memoria virtuale

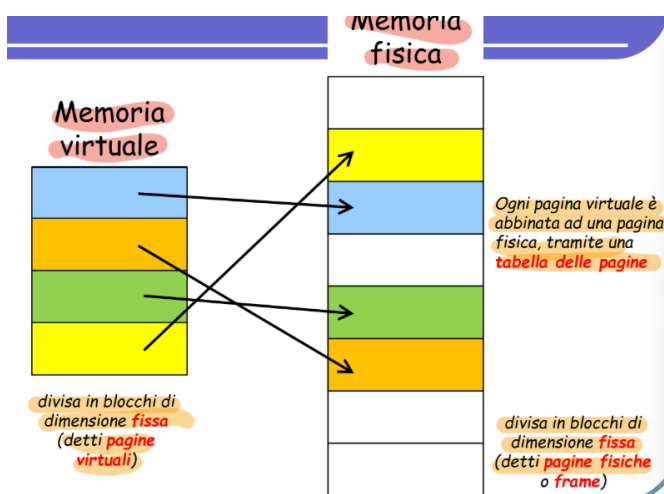
Ci sono due problemi:

- 1) Se c'è la memoria virtuale cosa succede?
- 2) Come realmente funziona la memoria cache.

Nei moderni sistemi di elaborazione, il Sistema Operativo sposta i programmi ed i dati dalla memoria secondaria alla memoria principale; le tecniche di spostamento prendono il nome di **MEMORIA VIRTUALE**. Di conseguenza, il Processore genera degli indirizzi virtuali che verranno poi tradotti negli indirizzi fisici mediante opportuni strumenti hw e sw.

I programmi e i dati sono organizzati in unità di lunghezza predefinita: le pagine. La tecnica di paginazione permette che lo spazio degli indirizzi fisici di un processo non sia contiguo.

La pagina non è altro che l'unità minima di informazioni che viene spostata dalla memoria secondaria alla memoria centrale (RAM).



La posizione (indirizzi) di codice e dati nella memoria del processo è una astrazione (indirizzi "virtuali"). La posizione effettiva in memoria fisica è gestita dal sistema operativo. La CPU vede indirizzi fisici che devono essere mappati su indirizzi fisici che si trovano in RAM.

La MMU (memory management unit) è un dispositivo hardware che consente durante la fase di esecuzione di associare gli indirizzi logici agli indirizzi fisici.

Le differenze principali tra cache e memoria virtuale sono due:

- 1) La cache opera sul principio di località per anticipare le richieste del processore
- 2) C'è una gestione concettuale e pratica totalmente diversa. Il cache miss è gestito totalmente dall'hardware. Per la memoria virtuale invece si ha un page fault ed è gestito dal software. Se il dato manca in memoria virtuale, e cioè si trova nel DISCO e non nella RAM, l'hardware cede il controllo generando un'eccezione. Il sistema operativo deve intervenire per recuperare il dato dal disco tramite driver e DMA.

La cache e la memoria virtuale lavorano inevitabilmente in sinergia e ciò significa che il sistema hardware della cache deve parlare con la memoria virtuale. Possiamo avere 4 situazioni:

- 1) **No miss/No fault:** il dato si trova già nella cache e quindi siamo apposto non richiede accessi ne alla RAM ne al disco. È la situazione ideale
- 2) **Si miss/No fault:** il dato non si trova nella cache, ma si trova fisicamente nella RAM. Si verifica un cache miss e quindi il processore deve caricare il dato dalla RAM alla cache.
- 3) **Si miss/si fault:** scenario peggiore perché il dato non è nella cache e non è neanche in RAM. Quindi viene sollevato un page fault che deve essere gestito tramite software. La pagina mancante viene caricata dal disco in RAM e a questo punto l'hardware riprende il controllo e carica il dato dalla RAM alla cache.
- 4) **No miss/si Fault:** questa situazione è impossibile perché se il dato è nella cache sicuramente è in RAM. La cache è per definizione un sottoinsieme della memoria principale; se il Sistema Operativo decide di "sfrattare" una pagina dalla RAM al disco, è obbligato per architettura a invalidare (o scaricare in memoria) tutte le linee di cache che contengono frammenti di quella pagina. Se non lo facesse, la cache conterrebbe dati "orfani" e si distruggerebbe la coerenza del sistema.

Un Indirizzo logico è diviso in due parti:

- **Chiave** che serve come indice per la tabella delle pagine che contiene l'indirizzo di base nella memoria fisica di ogni pagina.
- **Blocco:** è uno scostamento che combinato con l'indirizzo base definisce l'indirizzo della memoria fisica.

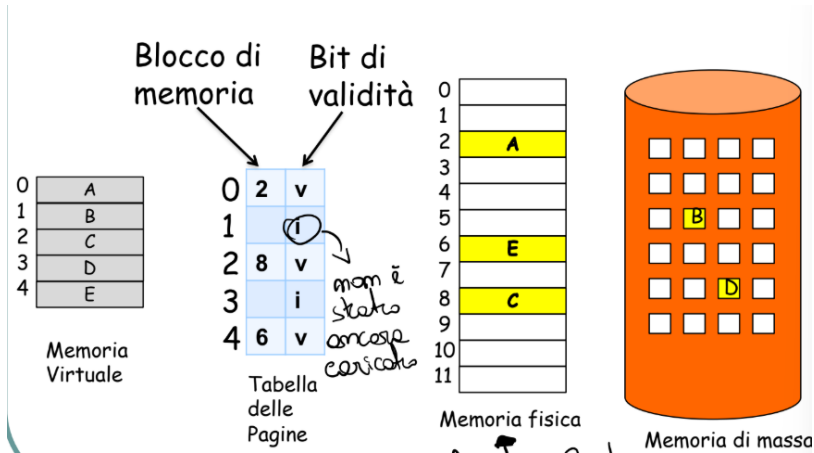
La trasformazione avviene facendo entrare la chiave in una tabella ed esce la chiave fisica cf. la memoria è sempre di tipo associativo. Se la chiave non c'è significa che c'è un page fault ovvero la pagina non è in memoria. Si genera un'interruzione che mi deve far leggere da un dispositivo la pagina.

Se invece la pagina è presente in memoria allora viene calcolato l'indirizzo fisico.

La tabella delle Pagine è in RAM ed è solitamente di grandi dimensioni.

la grandezza standard delle pagine è di 4KB ovvero 4096 byte. Se le pagine fossero più grandi potremmo introdurre frammentazione interna, se invece fossero più piccole, il SO dovrebbe gestire un numero spropositato di pagine.

MMU prende una chiave logica e la trasforma in una chiave fisica. È un rapporto 1 ad 1.



non tutte le pagine si trovano in RAM. Alcune potrebbero essere sul disco. Se il processo tenta di accedere a una pagina che non era stata ancora caricata in memoria si verifica un Page Fault ovvero un'eccezione.

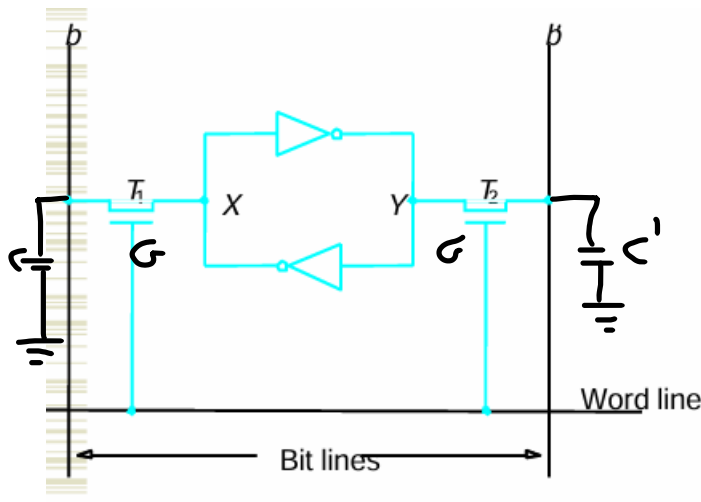
A questo punto si individua un blocco di memoria da sostituire, si trasferisce la pagina in memoria, si aggiorna la tabella delle pagine e si riavvia l'istruzione interrotta. Ovviamente la pagina sostituita non deve essere

sovrascritta, ma portata nel disco.

1.2.3 Memorie statiche e memorie dinamiche

Gli elementi di memoria statici utilizzano un bistabile ovvero un elemento che effettua una retroazione che consente di conservare il dato. Sono robuste e veloci. La memorizzazione si basa sulla presenza di un latch.

La cella di memoria è composta da un latch e due porte a logica di trasmissione. La gate è collegata alla word line. Inoltre sono presenti due capacità parassite ai lati delle bit line.

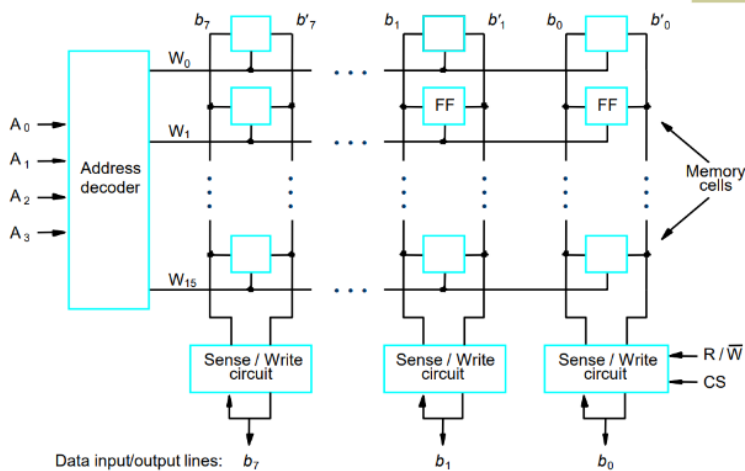


Esistono più tipi di memorie SRAM dovute al numero di NMOS utilizzati:

- 4T usa due NMOS per le porte e 2 NMOS per le porte NOT. Le porte NOT sono realizzate con un NMOS sulla rete di pull-down e con una resistenza sulla rete di pull-up. Queste resistenze sono realizzate con polisilicio che presenta una resistività molto alta a parità di occupazione di area rispetto alle resistenze classiche.
- 6T usano 6MOS e quindi la porta NOT è realizzata in logica CMOS.

La memoria è indirizzata tramite un decoder che consente di indirizzare 2^n locazioni dove n è il numero di bit dell'indirizzo. Ci sono poi 8 colonne per formazione il byte e quindi il contenuto della locazione.

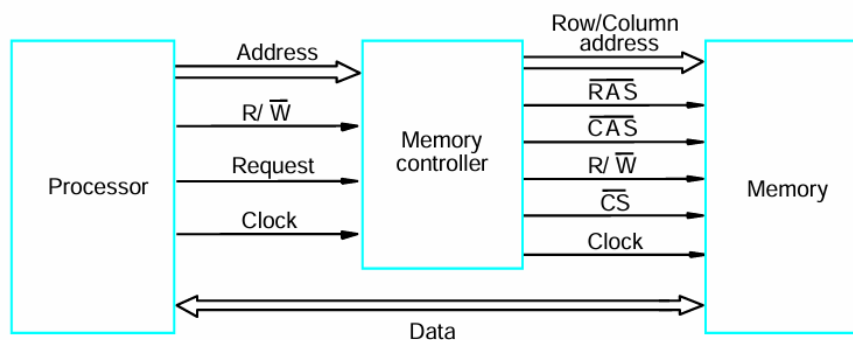
Organizzazione di una RAM statica (16parole x 8bit)



51

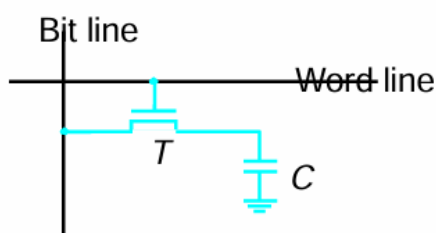
La memoria dinamica funziona con un effetto capacitivo, ovvero una capacità che se è carica mantiene l'1 logico, mentre se è scarica mantiene lo zero logico. Il vantaggio di avere una memoria ad effetto capacitivo è dato dal fatto che posso mettere più celle. Quelle dinamiche sono più lente perché l'effetto capacitivo fa perdere la carica. le memorie dinamiche presentano un enorme riduzione di spazio occupato perché hanno 1 solo transistor.

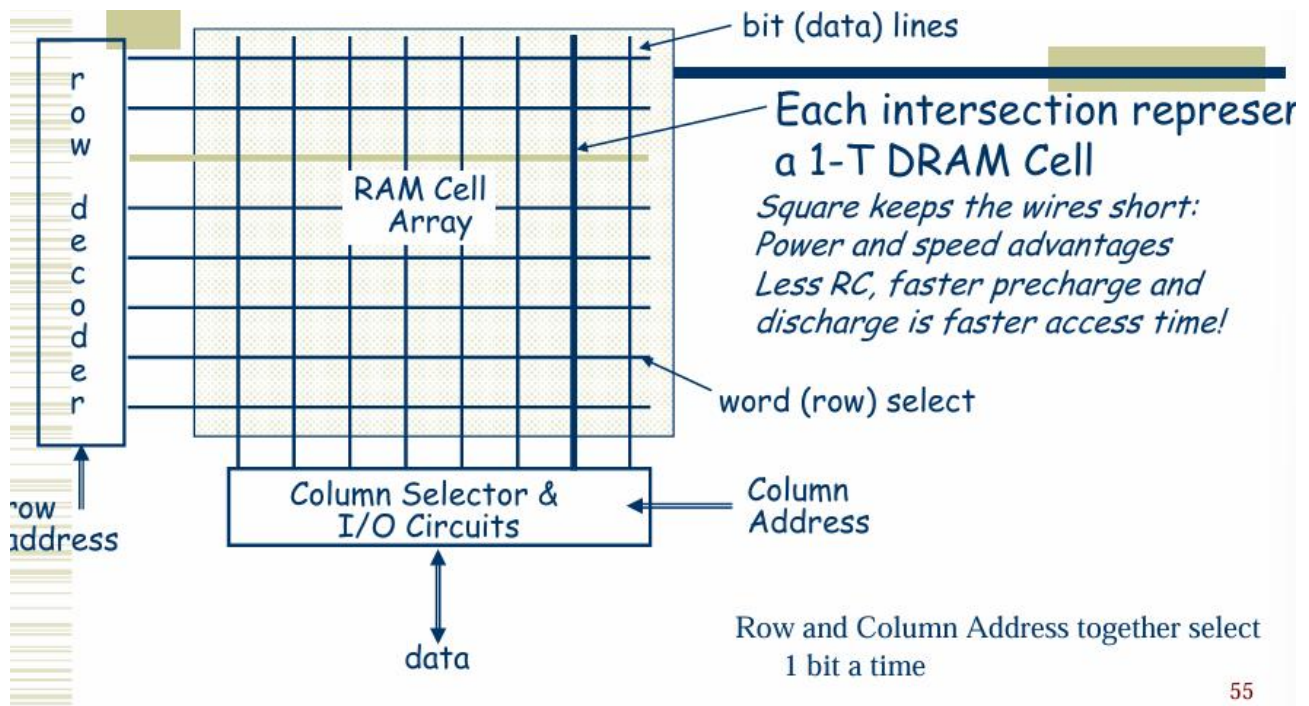
La memoria dinamica non si indirizza dando l'indirizzo, non è lineare. È organizzata in forma matriciale. Ogni bit è all'interno di una matrice.



usando n PIN.

il processore fornisce l'intero indirizzo al memory controller. Poi lui ha il compito di dividerlo per riga e colonna e successivamente li invia sugli stessi pin in due momenti diversi. Nelle SRAM invece l'indirizzo è inviato insieme in parallelo



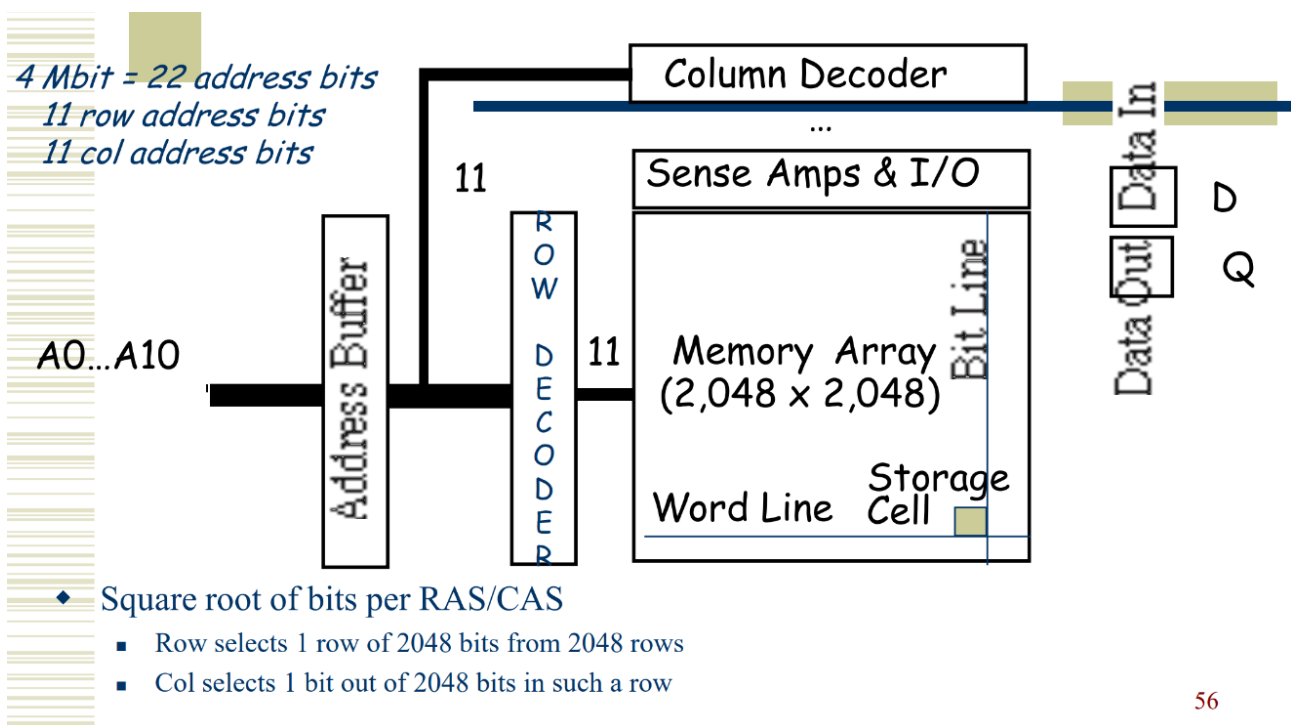


L'indirizzo che noi diamo 4228 in una memoria dinamica viene diviso in due parti:

- Una parte seleziona la riga e si chiama RAS
- Una parte la colonna e si chiama CAS.

42 prende la riga e poi con 28 prendiamo la colonna.

Questo implica che questa memoria deve ricevere prima l'indirizzo della riga e poi l'indirizzo della colonna. Il bus non mette tutto l'indirizzo ma prima una metà e poi l'altra.



Quando il segnale di RAS attiva una specifica riga, le cariche elettriche non vengono inviate direttamente sul bus, ma passano attraverso degli amplificatori e bloccato all'interno di un registro hardware interno al chip di memoria (Row buffer). Questo è fondamentale per implementare le letture in Burst (Page Mode), sfruttando il principio di località spaziale.

Se il processore necessita di leggere blocchi contigui di dati come avviene per il riempimento della cache, il memory controller invia il segnale RAS una sola volta per caricare l'intera riga nel Row Buffer interno.

Successivamente il controller si limita ad inviare una rapida sequenza di segnali CAS e ad ogni nuovo CAS, il MUX estrae i bit consecutivi direttamente dal registro interno, già pronto e amplificato, abbattendo i tempi di accesso.

Incastrando la memoria dinamica con i concetti di cache e memoria virtuale otteniamo che l'Indirizzo parte logico. Se ci sta MMU viene diviso in due parti (p e offset) ed entra in una prima tabella. Diventa indirizzo fisico. Questo viene proposto alla cache e viene diviso in un altro modo in base al tipo di cache. Se non lo trova va nella RAM che lo divide in due parti RAS e CAS. Tutto questo se la pagina è in memoria.

Se non è in memoria non arriva alla cache, perché dice che la pagina non c'è quindi è una conseguenza che non sta nella cache.

Nelle DRAM quindi la presenza o l'assenza di dato è dovuto alla presenza assenza di carica elettrica. La carica con il passare del tempo si attenua a causa delle correnti di dispersione. C'è bisogno quindi di cicli di refresh per mantenere il dato all'interno della memoria.

La tecnologia delle memorie tende a far sì che questo tempo sia il più possibile sovrapposto per poter continuare ad usare la memoria, perché quando facciamo il refresh la memoria non risponderebbe. Quando leggiamo e scriviamo NON c'è il refresh.

Per garantire che i dati non vadano persi il controller di memoria deve periodicamente rileggere ogni riga e riscrivere i valori, esattamente come se stesse facendo una lettura normale, in modo da ricaricare ogni condensatore prima che si svuoti troppo.

Le operazioni di lettura e scrittura non equivalgono e non corrispondono ad un ciclo di refresh. Non resettano i contatori di invecchiamento della matrice. È indispensabile la presenza di un circuito di refresh hardware.

Il ciclo di refresh è simile a quello di lettura ma viene svolto più velocemente perché:

- per il refresh si ha bisogno solo dell'indirizzo della riga e non della colonna.
- il dato letto non deve essere caricato sui data bus per essere mandato alla CPU

Deve avere due oggetti per poter funzionare: un contatore per sapere quale riga devo rinfrescare, è un contatore di riga per selezionare la riga. Serve un altro contatore, quello che misura il tempo per sapere dopo quanto tempo devo rinfrescare.

Il refresh va effettuato ogni $T/\text{num_righe}$ dove T è il tempo di decadimento

Nell'evoluzione delle memorie si è passato alla memorizzazione di byte e non di singoli bit. Prendiamo un indirizzo fatto di 21 bit:

- 12 bit vengono inviati al decoder di riga $2^{12}=4096$
- I restanti 9 vengono inviati al decoder di colonna $2^9=512$

Quindi ogni locazione individuata dalla linea decodificata per la riga e per la colonna non corrisponde ad un bit, ma ad un byte. Parliamo quindi di memoria dinamica da 2Mx8 bit e la divisione è 4096 x (512 x8)



Il grafico a sinistra mostra la lettura. Il Memory controller mette l'indirizzo di riga sul BUS ADDRESS e abbassa il segnale di RAS. Il chip di memoria cattura l'indirizzo e seleziona l'intera riga.

Dopo qualche istante inserisce l'indirizzo di colonna e abbassa il CAS. Dopo un breve ritardo sputa fuori i bit richiesti sul BUS DOUT.

Il grafico a destra invece è per la scrittura. dopo aver dato la riga e prima di dare la colonna, il memory controller abbassa il segnale di WRITE ENABLE per far capire che appena darà il CAS bisogna scrivere in quella locazione.

In entrambe le operazioni il RAS rimane sempre basso fino alla fine delle operazioni perché bisogna rimanere sempre attiva la riga. Appena si abbassa RAS, l'intera riga viene riversata nei SENSE AMPLIFIERS. Se abbassassi il RAS prima del tempo, la riga verrebbe scollegata dagli amplificatori.

1.2.4 Sviluppi Tecnologiche

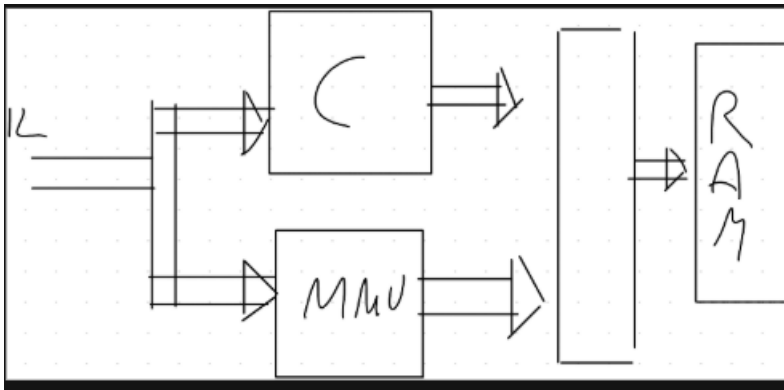
Per colmare il divario tra la velocità del processore e la lentezza della memoria principale, l'architettura delle DRAM si è evoluta nel tempo:

- EDO RAM:
- SDRAM: DRAM sincrone e segna una svolta perché inserisce nella memoria un'interfaccia temporizzata sincronizzata con il bus di sistema. Internamente utilizza diversi banchi che possono essere utilizzati in contemporanea.
- DDR SDRAM: è lo standard delle memorie moderne. Sfrutta sia il fronte di salita che il fronte di discesa del segnale di clock per il trasferimento dei dati.

1.2.5 Cache Virtuale

Virtual cache: la cache invece di prendere indirizzi fisici, prende indirizzi logici. Vede se ci sta quell'oggetto che ha quell'indirizzo logico. Quindi MMU lavora in parallelo alla cache.

Mettere la MMU dopo la cache rallenta il sistema perché se non trova l'indirizzo in modo sequenziale poi fa la conversione. Mettendolo in parallelo invece lo fa contemporaneamente. Poi può servirmi oppure no.



Se non avessi le memorie associative funzionerebbe la cache? La memoria virtuale? No perché mi consente di avere più elementi usando meno cose.

Facciamo una MMU con 4 pagine in memoria quindi 4 comparatori. Ogni pagina è di 1 megabyte. Quindi memoria virtuale di 4 megabyte.

La memoria virtuale in genere è grande.

La MMU quindi è un po diversa.

Un solo comparatore e poi ritorna il concetto di settore per poter avere più chiavi.

La chiave logica quindi avrà un pezzo che permette di vedere il settore e un pezzo il comparatore. L'indirizzo lo divido in 3 parti. Per poterlo implementare sicuro non può essere fatto con una chiave un comparatore.

Lo svantaggio della memoria cache però è che la dim è $K \cdot S \cdot L$. se lo faccio con $K=1$ devo fare tantissimi settori o blocchi lunghissimi.

La memoria ha due parametri fondamentali: Taccesso è il tempo in cui prendiamo un dato. Poi il tempo di ciclo è il tempo in cui possiamo prendere un altro dato.

Se migliora la tecnologia spero che migliorano entrambi.

Le dimensioni dei blocchi della cache sono minori di quelle delle pagine. Ne consegue che la possibilità di non trovare un dato nella cache è molto più alta della possibilità di non trovare un dato in una pagina che si trova in memoria.

Lezione 7 maggio Architetture parallele

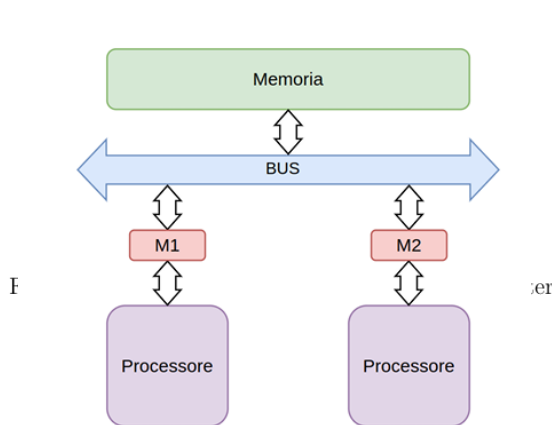


Figura 3.4: Sistema multicore

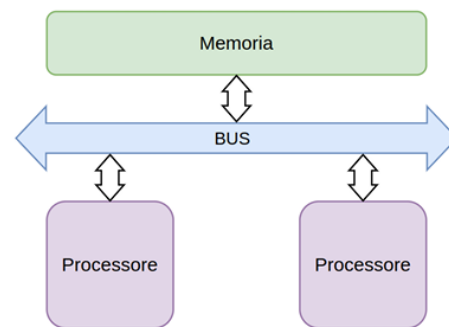


Figura 3.3: Sistema a memoria condivisa

L'idea di avere **più computer** replicati non risulta essere molto efficiente perché la comunicazione tramite dispositivi di I/O non è veloce come i processori per cui avremmo un rallentamento delle operazioni.

Per superare il problema potremmo utilizzare una memoria condivisa e un bus condiviso e replicare soltanto i processori (**multiprocessore**). In questo caso avremmo problemi di concorrenza nell'accesso al bus e alla memoria.

Potremmo quindi pensare di utilizzare una gerarchia di memorie che permetta di ridurre l'accesso alla memoria principale. Questi sistemi vengono detti **multicore**.

Perché ad un certo punto abbiamo costruito sistemi con gpu?

Fino ad ora abbiamo visto **sistemi caratterizzati da un solo processore**, dove cercavamo di **migliorare le prestazioni tramite tecniche hardware e nelle architetture superscalari più si aumentavano le pipeline e più le prestazioni aumentavano**.

Vogliamo andare sempre più veloci. Per andare più veloci **la pipe è una risposta che porta un parallelismo a livello di istruzioni ovvero posso avere 5 (nel caso di 5 stadi) istruzioni contemporaneamente**, una per ogni stadio. Il che comportava ovviamente dei problemi come:

- gestione dei salti → tecniche di branch prediction;
- conflitti sui dati → operand forwarding e successivamente internal forwarding con tabelle di ibernazione;
- gestione delle interruzioni → tecniche di breakpoint e history table

Ad un certo punto nel mondo dell'elettronica si è tentato di dire per andare più veloci dobbiamo aumentare la frequenza di clock.

Dire che un processore va a 1 GHz significa dire che in 1 secondo riceve 1 miliardo di impulsi di clock.

Più aumentiamo la frequenza e più nello stesso intervallo di tempo il processore può eseguire più istruzioni. In un primo momento il livello di integrazione aveva due parametri: tensione e frequenza. **Ad un certo punto non è stato più possibile aumentare la frequenza mantenendo la potenza sotto controllo**. La legge di

scalabilità detta anche scala Dennard (1974) affermava che man mano che i transistor diventavano più piccoli, la loro densità di potenza rimane costante, così che il consumo di energia rimanga proporzionale all'area.

La potenza dinamica è governata dall'equazione $C \cdot V^2 \cdot f$.

Intorno al 2005, i transistor erano diventati così piccoli che non si poteva più abbassare la Tensione per alzare la frequenza. Se l'avessero abbassata ulteriormente, la "pressione" non sarebbe bastata a distinguere in modo netto uno 0 da un 1 e la potenza sarebbe esplosa.

Essendo bloccati con una V costante (circa 1 volt), la formula letale è diventata evidente. Continuare ad aumentare la frequenza significava far aumentare in modo catastrofico il calore prodotto dal chip, superando la capacità fisica dei dissipatori di raffreddarlo.

- 1) Quando la tensione V è arrivata ad un limite da non poter essere più abbassata, la potenza ha iniziato a scalare in modo quadratico rispetto a V e in modo costante rispetto ad f generando un calore insostenibile. Quindi il primo problema è stato un **problema di tipo energetico**. (POWER WALL)
- 2) Il secondo problema è che sistemi che lavorano con la pipe hanno un problema di gestione della pipe come quelli citati in precedenza. Questo sistema ha iniziato ad avere dei limiti importanti. (ILP WALL)
- 3) Il terzo limite è che c'è una forte dipendenza dalla memoria. (MEMORY WALL)

Dobbiamo utilizzare non più un solo elemento ma di più, verso forme di parallelismo.

Il limite dell'ILP (instruction-level parallelism) e la fine del Dennard scaling hanno portato l'industria dell'informatica a progettare i chip multi-core.

Bisogna sviluppare delle architetture diverse tra loro in cui il principio primo è la pipe. Esistono differenti tassonomie. Alcune risentono del tempo in cui sono state date. La prima tassonomia di Flynn. Le architetture viste fino ad ora sono certamente architetture SISD ovvero singola istruzione singolo dato.

Tassonomia di Flynn:

- MIMD: eseguono istruzioni diverse su dati diversi. Considerando architetture formate da più processori che lavorano su dati diversi si distinguono in due grandi famiglie:
 - Sistemi multiprocessore: più unità di elaborazione che lavorano e interagiscono tramite una memoria condivisa M.
 - Sistemi multicomputer: i processori comunicano scambiando messaggi. Ogni processore è dotato di un sistema di I/O grazie al quale può comunicare con gli altri.
- SIMD
- MISD
- SISD

Queste architetture rispondono a criteri differenti.

Il 68k è singola istruzione singolo dato. Più difficile è trovare MISD cioè fanno istruzioni diverse sullo stesso dato. Possono servire per problemi di fault tolerance.

Inoltre c'è una categoria aggiuntiva ovvero SPMD single program multiple data. Il parallelismo non è a livello di istruzione ma di programma.

Ci sono delle estensioni moderne alla tassonomia di Flynn:

- **SPMD single Program, Multiple Data**: lo stesso codice esegue operazioni su nodi distinti, ciascuno dei quali opera su una porzione di dati diversa.
- **SIMT single instruction, Multiple Threads**: introdotto da NVIDIA con CUDA è una via di mezzo tra SIMD e MIMD.

2.1 Architetture SIMD



Una sola istruzione viene fetchata e decodificata, poi distribuita in broadcast a N lane parallele.

Il valore di N varia da 4 (SSE a 128 bit, 32-bit float) fino a 16 (AVX-512).

L'efficienza energetica nasce proprio qui: il fetch/decode è ammortizzato su tutti gli elementi

Ogni lane possiede una propria ALU e opera su un proprio registro. Le operazioni supportate sono aritmetiche, logiche.

Ora vogliamo discutere le architetture SIMD ovvero singola istruzione multiplo dato **quindi operante su più dati come vettori o matrici.**

$$N = 100 \\ \sum_{i=1} a_i b_i$$

Questa istruzione ci dice che dobbiamo fare la stessa operazione (moltiplicazione) tante volte su dati diversi. La scelta corretta è un'architettura SIMD.

In realtà potremmo usare anche un'architettura SPMD perché possiamo dividere la sommatoria in 10 sommatorie e quindi 10 programmi. Il programma è sempre lo stesso ma ho dati differenti.

Il **modello è sempre lo stesso ovvero la pipe IF->ID->EX->MEM->WB**

Perciò per non perdere tempo potremmo evitare ad ogni iterazione di prelevare l'istruzione, decodificarla e poi eseguirla, ma piuttosto la prendo e decodifico una volta sola così faccio la stessa operazione su dati diverse avendola decodificata una sola volta. Modello SIMD prendo l'istruzione una sola volta la faccio su dati differenti. La difficoltà è che questo sistema richiederà un flusso di dati molto più elevato. Questo è quello che fanno le GPU. Fanno sempre la stessa operazione. **Nascono per la multimedialità perché prendono un'immagine la digitalizzano e fanno una serie di operazioni veloci.**

Quando si sono costruiti i **processori potevamo fare l'addizione su 64 bit. Ma 64 bit per la multimedialità sono tantissimi, nessun occhio può distinguere 2^64 colori diversi. Invece di fare un'operazione su 64 bit puoi fare 4 operazioni uguali su 16 bit. Non hanno complicato troppo la fase esecutiva e hanno deciso di fare la stessa operazione su dati diversi. Quindi invece di fare un'addizione su 64 bit ne faccio 4 su 16 bit. Questo agli inizi degli anni 90. Questa intuizione è nata proprio per la multimedialità: l'intel l'ha battezzata inizialmente MMX (multiMedia extensions) sui processori Pentium per poi evolverla nelle istruzioni SSE e oggi nelle AVX**

Poi hanno pensato anche di metterne di più di queste unità. Non ha risolto il problema di flusso di dati perché i dati devono entrare velocemente nel sistema. I dati entrano meno velocemente di quanto il sistema riesce ad elaborarli.

Velocizziamo i programmi ma quali? Quelli fatti di tutte operazioni simili. Con i sistemi multimediali i dati non sono tantissimi all'inizio non erano usate. Con l'AI questi requisiti si sono alzati e per puro caso le GPU che servivano solo per la grafica sono state usate per i calcoli AI.

Questo sistema non va bene per fare ad esempio prenotazioni di un volo. Questi sistemi sono poco programmabili ovvero se volessi fare un'operazione diversa come la sommatoria della somma di a e b non andrebbero bene. Sono visti come acceleratori.

Quindi è importante che il programma venga scritto già in ottica di tale sistema, oppure il compilatore deve cercare di estrarre il parallelismo.

2.2 Architetture MultiCore e Multithreading

Se invece volessimo avere un sistema gestionale, ovvero tantissime richieste in contemporanea, sono necessarie più unità elaborative in modo che ogni richiesta venga gestita da ognuna. Queste unità sono in numero finito quindi ad un certo punto se arrivano più richieste di quante unità ho non posso accettarle. Allora si fa in modo che mando più richieste sulle stesse unità.

Quindi ho più core che possono eseguire più attività detti thread. È necessario che i processori abbiano tanti core, ma che ognuno sappia gestire in maniera efficiente i thread. Quando uno ad esempio fa un'operazione di I/O viene sostituito per non lasciare inutilizzato il core, perciò è necessario saper fare il cambio di contesto da uno all'altro (Program counter, registro di stato etc). questi sono i processori multicore e multithreading

Potrei utilizzare un thread per intero e poi passare a quello successivo, ma così facendo se uno fa un'operazione di I/O non starei poi sfruttando il core allora posso sostituirlo in favore di un altro in modo che il core faccia sempre qualcosa.

Avendo più unità, queste dovranno comunicare, ad esempio per scrivere in memoria, oppure su un DB. Va perciò mantenuto uno stato coerente.

Potremmo usare la mutua esclusione oppure il concetto di timestamp ovvero ad ogni thread è affidato un quanto di tempo e rifiuto la richiesta 12 se la 11 non è terminata ad esempio.

Questa seconda strada ha portato allo sviluppo delle architetture MIMD, poiché faccio istruzioni differenti su dati differenti.

Viste le due architetture (SIMD E MIMD) per andare ancora più veloce posso fonderle. Le GPU di oggi hanno tante unità elaborative (MIMD) ognuna fatta da architetture SIMD.

Se la GPU ha 4 moduli ognuno SIMD, complessivamente è SIMD o MIMD? È MIMD perché ogni unità potrebbe potenzialmente fare qualcosa di diverso.

Data un'architettura MIMD è difficile valutare quanto più veloce posso andare perché dipende da quanti thread ci sono, cosa fanno etc, ma dato un programma è possibile valutare quanto più velocemente può essere eseguito su un'architettura parallela.

2.2.1 Legge di Amdahl

Dato un programma P e quindi un tempo di elaborazione T_p abbiamo due tempi:

- 1) **tempo sequenziale T_s** : tempo di esecuzione relativo alla parte di codice sequenziale e quindi non parallelizzabile.
- 2) **tempo parallelizzabile T_s^{PAR}** : il tempo di esecuzione della parte di codice, che potrebbe essere parallelizzata, su una macchina sequenziale. Perciò può essere ottimizzato.

Se consideriamo la sommatoria di $A_i \cdot b_i$ tutti i prodotti sono parallelizzabili, mentre le somme no, cioè i singoli prodotti li posso fare in parallelo, ma poi devo sommarli e quindi ho bisogno di averli tutti. In realtà potrei un po' parallelizzare se divido la sommatoria dando ad esempio 10 prodotti a 10 unità ed ottenere delle somme parziali.

Se ho n unità devo definire il concetto di speed up: mostra l'incremento di velocità, nell'eseguire la parte parallela su un'architettura parallela piuttosto che su un'architettura sequenziale. Ovvero quanto potrei andare più veloce se eseguiessi in parallelo una parte? E per farlo ho bisogno di poterlo fare su un'architettura parallela.

T_s^{PAR} : tempo sequenziale che potrebbe essere parallelizzato se eseguito su un'architettura sequenziale

T_p^{PAR} : tempo di esecuzione parallelo su architettura parallela

$$\text{Speed up} = \frac{T_s^{PAR}}{T_p^{PAR}}$$

$$T_p^{PAR} = \frac{T_s^{PAR}}{N} \quad \text{dove } N \text{ è il numero di unità. Queste unità dovranno comunicare e quindi c'è un tempo di interazione } T_{int}$$

$$\text{SPEEDUP} = \frac{T_S^{\text{PAR}}}{\frac{T_S^{\text{PAR}}}{N} + T_{\text{INT}}} = \frac{N \cdot T_S^{\text{PAR}}}{T_S^{\text{PAR}} + N \cdot T_{\text{INT}}} = \frac{\cancel{T_S^{\text{PAR}}} \cdot N}{\cancel{T_S^{\text{PAR}}} \left(1 + N \cdot \frac{T_{\text{INT}}}{T_S^{\text{PAR}}}\right)} = \frac{N}{1 + N \cdot \frac{T_{\text{INT}}}{T_S^{\text{PAR}}}}$$

Abbiamo detto che $\frac{T_S^{\text{PAR}}}{N} = T_p^{\text{PAR}}$

$$\text{SPEEDUP} = \frac{N}{1 + \frac{T_{\text{INT}}}{T_p^{\text{PAR}}}}$$

Se T_{INT} è piccolo posso usare le serie di Taylor della funzione $\frac{1}{1+x} = 1 - x + x^2 - x^3$

$$N \left(1 - \frac{T_{\text{INT}}}{T_p^{\text{PAR}}}\right)$$

Andiamo effettivamente N volte più veloci se il tempo di interazione è molto piccolo oppure a parità di tempo di interazione devo aumentare il T_p^{PAR} ovvero calcolare di più. Facciamo molto calcolo e interagiamo poco. Se invece è viceversa allora non va bene ovvero il tempo di interazione è alto in merito al tempo di calcolo e allora lo speed up si abbassa notevolmente.

$\sum_{i=1}^{100} a_i b_i$	$\sum_{i=1}^{1000} a_i b_i$
----------------------------	-----------------------------

Chi va più veloce la sommatoria da 1 a 100 o da 1 a 1000? Quella più efficiente è quella che va da 1 a 1000 poiché ogni nodo fa 250 operazioni e poi comunica con gli altri. L'altra sommatoria invece fa 25 calcoli e poi comunica quindi il tempo di interazione impatta maggiormente nella formula.

Per la prima sommatoria ogni processore fa 250 operazioni T_p^{PAR} è molto grande. $T_{\text{INT}}/T_p^{\text{PAR}}$ è molto piccolo e quindi speed up si approssima ad N .

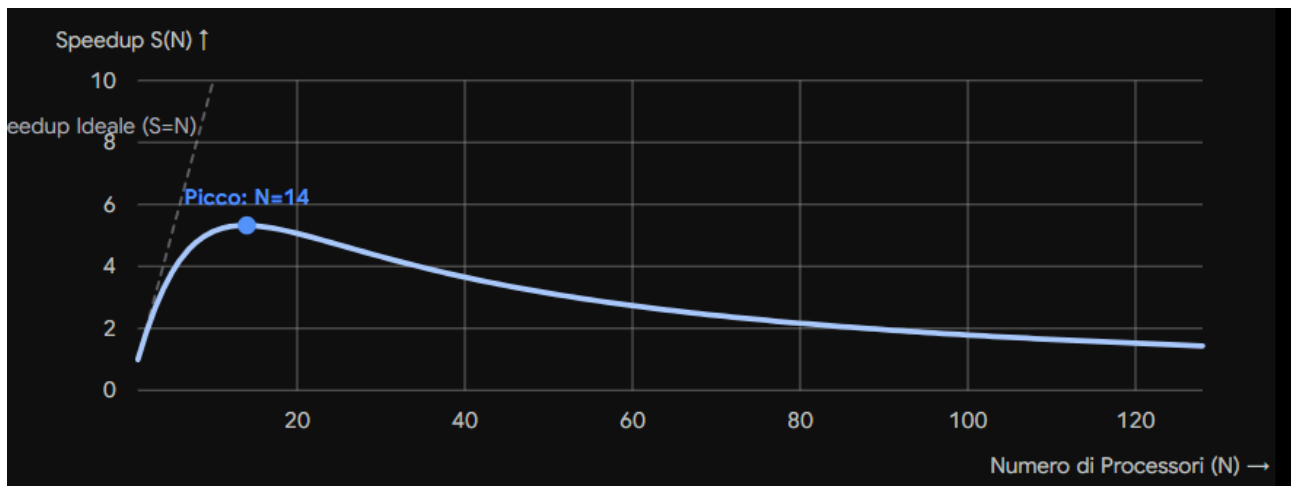
Nel caso della sommatoria da 1 a 100 T_p^{PAR} è inferiore e quindi il rapporto è più alto facendo abbassare lo speed up.

Se il tempo di interazione fosse pari a 0 allora avremmo che per entrambi lo speed up è uguale a N .

Se estendiamo questo concetto alle GPU, inizialmente non hanno avuto tanto successo perché lo speed up era basso. Con l'IA è aumentato il T_p^{PAR} e quindi speed up alto.

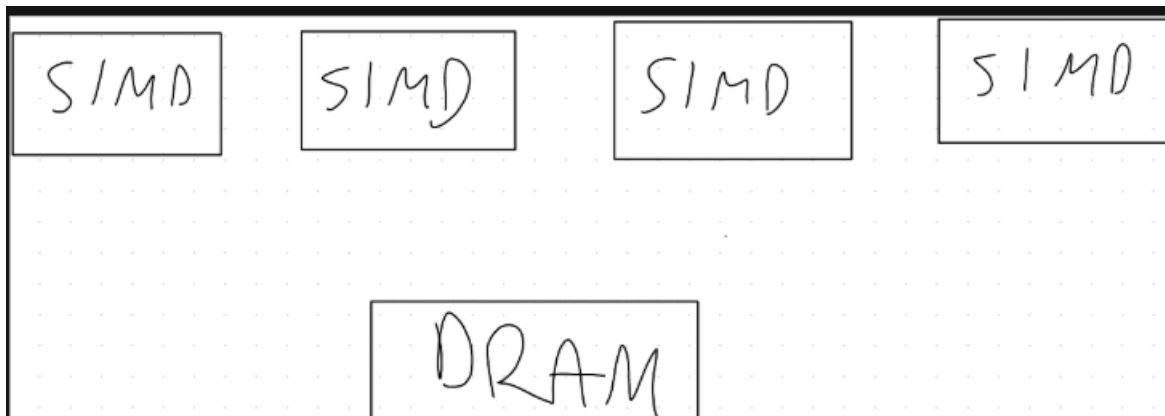
Un altro parametro importante è l'efficienza definita come il rapporto tra speed-up e numero di processori. Il valore ideale sarebbe =1 raggiungibile solo se $T_{\text{INT}} = 0$. Questo limite però non è ottenibile: qualunque programma reale ha una componente di interazione non azzerabile.

Più nodi non sono sempre meglio.



Se N tende a infinito, il numero

Consideriamo una GPU composta da 4 moduli SIMD



C'è anche una memoria che contiene i dati da elaborare e l'operazione che voglio fare (la sommatoria). Tale operazione può essere decomposta in tanti sotto prodotti: p_0, p_1, p_2 e così via.

Se ho 4 moduli sembrerebbe logico dividere l'operazione in 4 grandi blocchi, uno per modulo. Questo non è ottimale perché il tempo di interazione cresce.

Ogni thread farebbe:

- chiedo dato
- aspetto
- calcolo
- chiedo dato
- aspetto etc

il singolo modulo ad esempio ha 8 lane e quindi prendo i dati per 8 prodotti il modulo subito fa gli 8 prodotti e quindi devo andare a chiedere di nuovo altri dati. in questo tempo il modulo è fermo. per questo diciamo che c'è bisogno di un elevato flusso di dati.

se ogni modulo ha un solo thread, quando quel thread aspetta la memoria, il modulo è completamente fermo.

La soluzione è dare ad ogni modulo molti thread (n per modulo), in modo che mentre alcuni thread aspettano i dati, altri stanno calcolando e il modulo non si ferma.

Quindi divido in $k \gg 4$.

Usiamo comunque il concetto di multithread ma questa volta non per fare cose differenti ma per fare la stessa operazione. Questo meccanismo riduce al minimo il tempo di interazione perché mentre alcuni aspettano, altri eseguono e quindi si sovrappongono i tempi.

Tale sistema migliora la latenza o la banda? Sicuramente non migliora la latenza perché il tempo che un thread aspetta il suo dato rimane lo stesso, ma la banda perché ci sono tanti thread che continuamente chiedono dati alla memoria.

Un meccanismo simile a quanto visto per i thread era già stato introdotto ovvero per l'internal forwarding.

La logica è la stessa, se sto aspettando un dato dalla memoria, iberno e passo all'istruzione dopo senza bloccare totalmente la pipe.

RICAPITOLAZIONE

Calcolare: $\sum (a_i \times b_i)$ per $i = 1..1000$

Cioè: $a_1 \times b_1 + a_2 \times b_2 + a_3 \times b_3 + \dots + a_{1000} \times b_{1000}$

FASE 1: Singolo modulo SIMD con 4 lane

Questo modulo fa 4 moltiplicazioni contemporaneamente.

Ciclo 1: carica a_1, a_2, a_3, a_4 e b_1, b_2, b_3, b_4 dalla RAM → aspetta...

poi calcola: $[a_1 \times b_1, a_2 \times b_2, a_3 \times b_3, a_4 \times b_4]$

e così via per 250 cicli.

FASE 2: ho 4 moduli SIMD senza thread

Divido il lavoro: ogni modulo prende 250 elementi.

Modulo 0: calcola $a_1 \times b_1 \dots a_{250} \times b_{250}$

Modulo 1: calcola $a_{251} \times b_{251} \dots a_{500} \times b_{500}$

Modulo 2: calcola $a_{501} \times b_{501} \dots a_{750} \times b_{750}$

Modulo 3: calcola $a_{751} \times b_{751} \dots a_{1000} \times b_{1000}$

Ogni singolo modulo si comporta come prima quindi chiede i dati per le 4 moltiplicazioni e nel mentre aspetta, poi calcola e richiede gli altri dati e così via. Tutti e 4 i moduli sono fermi per la maggior parte del tempo.

FASE3: 4 moduli SIMD con thread

Invece di dividere in 4 blocchi grandi divido in $k=100$ blocchi di cui ogni blocco da 10 elementi. Essendo 100 blocchi, e 4 unità, devo dare 25 blocchi ad ogni unità e allora assegno ad ogni unità 25 thread.

Questi thread lavorano insieme, ognuno con il compito di dover fare 10 prodotti. Il vantaggio è che mentre alcuni thread aspettano la memoria, altri calcolano.

Nelle architetture Packed SIMD le singole "lane" non hanno alcun accesso autonomo alla memoria. Non ci sono 8 richieste di memoria separate.

Esiste un'unica unità a monte, la Load/Store Unit (LSU), che comunica con la cache/memoria principale. Quando il processore deve alimentare quelle 8 lane, esegue un'istruzione di Packed Load.

I blocchi contrassegnati come $R_0 \dots R_7$ non sono registri indipendenti. Sono fisicamente "slice" (fette) di un unico, grande registro hardware (es. un registro YMM da 256 bit).

La Load/Store Unit prende il blocco da 256 bit appena letto dalla memoria (che contiene i primi 8 elementi dell'array $\$A\$$) e lo scrive "in un colpo solo" in questo grande registro. Subito dopo fa lo stesso per i primi 8 elementi dell'array $\$B\$$, mettendoli in un secondo registro SIMD.

il DMA è usato sia nei multicomputer perché bisogna scambiarsi messaggi con dei driver, con degli oggetti esterni. devono essere efficienti nel copiare i messaggi in memoria. il DMA è usato anche nei multiprocessore per copiare blocchi di memoria.

dove non è usato? sicuramente nella coerenza della cache e nella cache miss perché si usano meccanismi hardware.

nel page fault il DMA si può usare per trasferire le pagine dal disco alla memoria.